Python

>It is a high level, general purpose, open source,interpreted programming language created by
Guido van rossum and first released in 1991.

>It is widely used in AI & Automation due to
easy to understand
less coding time
huge lib support

>Only downside of python is it's slow execution

>It was named Python after a comedy show Monty Python's Flying Circus

Python Installation

>We need to download python setup from python.org
>current version(10th feb 2026) is 3.14.3
>Following componenets are bundled with setup
1.Interpreter--->responsible to execute python code(handle transalation high to low level)
2.IDLE---------->editor to write & run python code
3.Docs---------->official reference of python
4.Library--------->set of readymade functions
etc.

Numbers in Python

1.Integer(int)
>number with out fraction
>+ve or -ve
>theortically it is unlimited but practically depends on RAM size
real life exp:
---
count of students/emps/etc.
runs/wickets in cricket
otp
account no
etc.

2.Float(float)

>number with fraction
>+ve or -ve
>accuracy level upto 14-16 decimal places
>becomes infinite(inf) after 308 digits

real life exp:
---
strike rate
interest rate
gold weight
gravity
temperature
avg marks
etc.

Text in Python
==========
>it is called string(str)
>represented in 4 ways

A. '.....'
B. "....."
C. '''.....
.....
'''
D. """.....
.....
"""

real life exp:
person name,place name,address,pan,txn id,empid,order id,story,etc,

Escape Sequences

>are special chars having special meanings in output
>following are the commonly used escape chars

\n ----> nextline
\t ---> tab(0 to 8 spaces)
\u ----> unicode
etc.

How to ignore escape seqs

1st way:use raw string

r'.....\t...\n.....' #best suitable if need to ignore all escape seqs

2nd way:use escape char

'.....\\n.....\\t.....' #best suitable if need to ignore specific escape char

WAP to print following output using 4 print
o/p
rohit virat
surya hardik

print()

>it is a library function that writes 0-N values of any type to standard output(shell/terminal/cmd/etc.)

syntax

print(value1,value2,value3,.....sep=" ",end="\n")

sep

>it is a keyword argument in print() that separate values in output
>default value is space and we may also change it's value
>it must be string and can not be repeated

end

>it is a keyword argument in print() that controls end of print
>default value is \n and we may also change it's value
>it must be string and can not be repeated

Language Fundamentals(tokens)
==========
1.Keywords
2.Comment
3.Variable
4.Operators
5.Type Casting

6.User Input
7.Decision Making
8.Loop
9.String Handling
10.Container datatypes

Keywords

>are reserve words having special meanings
>we can not modify their meanings & create our own keywords
>there are 35 keywords(3.14),32 are in lower and 3 in capitalize

Diff b/w function and keyword
---

function	keyword
we may create custom fun	we can not create custom keyword
we may modify definition	we can not modify definition

Comment

>it is non-executable part of a program and used to improve readability of source code.
>python supports only single line comment,represented by #

```
#.....
```

Note:officially there is no concept of multiline comment

Variable

>A variable is a name used to identify and store a value in a program so it can be reused.
>It may change it's value in the further stmts of a program

syntax

```
varname=value
```

Note:Python is a dynamically typed language it means we need not to declare datatype of a variable and

it is done by language itself.

exp:

statically typed:

int age=25

C/C++/Java/etc.

dynamically typed:

age=25

ABC/Python/R/etc.

Rules:

A.keywords are not allowed as identifier(varname)

B.symbols(except underscore) are not allowed

C.digits are allowed but not as first char

D.space,escape sequences(\t,\n,..) are not allowed

E.min length=1,max length=no limit

Note:from memory point of view,a variable is a reference that points to (refers to) a value(object).

id()

>It is a library function that returns address of value refered by a variable

address=id(variable)

type()

>It is a library function that returns datatype of a value refered by a variable

datatype=type(variable)

Note:

>variables share address of same value to optimize memory usage

>int/float/str/bool values are immutable hence they are changed to a different memory block

Operators

=========

>operators are special symbols & keywords used to perform different types of operations like arithmetic,
comparision,logical,searching,deleting etc. on operands.

>operand is the data in form of either value or variable or expression.

>based on no. of operands,there are 3 categories of operators

1.Unary ---> works with 1 operand
2.Binary ---> works with 2 operands
3.Ternary ---> works with 3 operands

Commonly used binary operators

A. Arithmetic operators
B. Assignment operators
C. Comparision(Relational) operators
D. Logical operators
E. Bitwise operators
F. Membership operators
G. Identity operators

Commonly used unary operators

A. unary plus sign
B. unary minus sign
C. logical not

Commonly used ternary operators

A. conditional operator

Arithmetic operators

+	Arithmetic Addition
-	Arithmetic Subtraction
*	Arithmetic Product
**	Arithmetic Exponent
/	Arithmetic true division
//	Arithmetic floor division
%	Arithmetic modulus division

Precedence & Associativity

Precedence	Associativity
()	left to right
**	right to left
/,//,%,*	left to right
+,-	left to right

round()

>it is a library function that can be used to round a float value in following 2 ways

A. float to int

```
int_data=round(float_data)

if fraction part is
        >.5     round to next int

if fraction part is
        <.5     round to same int

if fraction part is
        =.5
        if decimal part is even round to same int
        if decimal part is odd  round to next int
```

exp:

```
5.6     --->    6
5.4     --->    5
5.5     --->    6   (as decimal part is (odd) 5)
6.5     --->    6   (as decimal part is (even) 6)
```

B. float to float to limit decimal places

```
float_data=round(float_data,decimal_places)
```

exp:

```
round(3.145789,2)     ---->     3.14
```

Assignment Operators

>it is used to define or update a variable

Simple Assignment

---

var=value/var/expression

Multiple Assignment

---

var1,var2,var3...=value1,value2,value3,...

Chained Assignment

---

var1=var2=var3=.....=value

Compound Assignment

---

var+=value	means	var=var+value
var-=value		
var*=value		
var**=value		
var/=value		
var//=value		
var%=value		

Comparision(Relational) Operators

>these operator return True or False after comparing 2 values.

>widely used in decision making & sorting

Types:

A.	equal comparision	==
B.	not equal comparision	!=
C.	greater comparision	>
D.	lesser comparision	<
E.	greater or equal comparision	>=
F.	lesser or equal comparision	<=

Logical Operators

>these operators also return bool result(True or False) by combining 2 bool values

>widely used in decision making with 2 or more comparisions

Types

A. logical and

True and True--------->True
True and False-------->False
False and True-------->False
False and False------->False

B. logical or

True or True---------->True
True or False--------->True
False or True--------->True
False or False-------->False

C. logical not(unary category)

not True--------------->False
not False-------------->True

Bitwise Operators

In python,bitwise operators are used in following specific cases

A. IOT/Embedded coding
B. Set theory implementation(union,intersection,..)
C. Masking(Filtering rows) in numpy & pandas libs
D. Chaining in langchain framework(GenAI)
E. Faster execution(left right shift) than arithmetic

etc.

Types

A. bitwise and (&)
B. bitwise or (|)
C. bitwise xor (^)
D. bitwise leftshift (<<)
E. bitwise rightshift (>>)

decimal	binary
0	00
1	01
2	10
3	11

4 100
5 101
6 110
7 111
8 1000
.
.

bitwise and (&)
---

1 & 1-------->1
1 & 0-------->0
0 & 1-------->0
0 & 0-------->0

bitwise or (|)
---

1 | 1-------->1
1 | 0-------->1
0 | 1-------->1
0 | 0-------->0

bitwise xor (^)
---

1 ^ 1-------->0
1 ^ 0-------->1
0 ^ 1-------->1
0 ^ 0-------->0

bitwise leftshift(<<)
---

a<<b ------>shift the bits of a to left by b positions(a will be doubled for each 1 left shift)

bitwise rightshift(>>)
---

a>>b------>shift the bits of a to right by b positions(a will be floor divide by 2 for each 1 right shift)

Type Casting
==========

>It is a technique to convert a datatype into another
Note:Actual type is never converted rather a new copy is created

2 types

A. Explicit type casting---->programmer is responsible
B. Implicit type casting---->language is responsible

str to float(explicit)
---
float_data=float(str_data)

float to str(explicit)
---
str_data=str(float_data)

str to int(explicit)
---
int_data=int(str_data)

int to str(explicit)
---
str_data=str(int_data)

float to int(explicit)
---
int_data=int(float_data)

int to float(implicit)
---
float_data=float(int_data)

boolean to int(implicit)
---
int_data=int(bool_data)
True--->1
False-->0
etc.

Note:type casting fails if source & target types are not compatiable
exp: '4.5k'------>float----->fails
'45k'------->int------->fails

User Input

=========

>It is a technique to read value from user during execution of program

```
str_data=input('prompt')
```

Note:input() always return user data as str and if we need to perform required operations then we have to use type casting.

user input as int

```
int_data=int(input('prompt'))
```

user input as float

```
float_data=float(input('prompt'))
```

Conditional Statements
==========

>are those stmts that are executed when a given condition is True or False

syntax-1

```
if bool_expr:          #code to be executed when it is True
    code
else:                  #code to be executed when bool_expr is False
    code
```

syntax-2:(nested/inner condition)

```
if bool_expr:
    code
    if bool_expr:
        code
    else:
        code
else:
    code
    if bool_expr:
        code
    else:
        code
```

syntax-3

```
if bool_expr:
        code
elif bool_expr:
        code
elif bool_expr:
        code
.
.
else:   #optional
        code
```

Note:
>next condition is only executed when previous evaluates to False
>if all conditions evaluate to False then else block is executed

Block in Python:

>A block represents group of stmts
>there are 2 approaches
1. Indented block
2. Inline block

syntax of indented block(with if):

```
if bool_expr:
        code
```

syntax of inline block(with if):

```
if bool_expr:single stmt
```

Note:if you want put multiple stmts then use ; after previous stmt

exp:
```
if bool_expr:stmt1;stmt2;stmt3;.....
```

>Inline block is generally used as short hand for 1 line block

Ternary Operator

>it was added in python 2.5 version to write if-else in a single statement
syntax:
true_value if condition else false_value

>we can not use elif directly with ternary operator but same can be implemented using nested ternary
syntax:
true_value if condition else (true_value if condition else false_value)

>it is widely useful in comprehension

Complex datatype

>it is used to represent a value in the form of
a+bj

where
a---------->real part(int/float)
bj---------->imaginary part

b---->real
j----->imaginary unit

>it is widely used in IOT & Embedded applications to represent circuit

keywords
if
else
elif

operators
arithmetic(+,-,*,**,/,//,%)
compare(>,>=,==.!=,<=,<)
logical(and,or,not)
bitwise(&,|,^,<<,>>)
ternary
assignment

functions

print()

round()
input()
int()
float()
type()
str()
abs()
etc.

Loop
=====
>A loop provides facility to execute code of block repeatedly at same point in program.
>Python provides 2 implementations of loop
>while loop
>for loop

syntax of while:

```
while bool_expr:
    code that you want to repeat
```

while-else

```
while bool_expr:
    code that you want to repeat
else:
    code that you want to run for False condition
```

Note:else is not executed when loop exits using break

continue & break keywords:
---
continue:it is used to exit/skip only current iteration
break:it is used to exit loop

>both can not be used outside loop
>generally used with if condition

for loop
=======
>it is used to iterate elements of an iterable one after other
>it is also called for each loop

syntax

```
for variable in iterable:
        code
```

syntax with else

```
for variable in iterable:
        code
else:           #executed when loop is completed after iterating all elements of iterable
        code
```

diff b/w while and for
---

while	for
1. best suitable when we don't know excat no. of iterations	1. best suitable when we know excat no. of iterations
2.it is condition based	2.it is iterable based

range()

>It is a python library fun that creates a sequence(iterable) of int values with symmetric gaps

```
seq=range(start,stop,step)
```

start--->inclusive
stop--->exclusive
step

```
+ve     --> forward seq
-ve     --> reverse seq
0       --> error
```

>we may also use range with default values

```
range(start,stop)      default step=1
range(stop)     default start=0,step=1
```

String Handling
==========

>String is a sequence of immutable(unmodifiable) unicode characters

>We can represent string in following 4 ways

A. '.....'
B. "....."
C. '''.....'''
D. """....."""

Operations

1.concatenation

>creating a new str by combining 2 strs

```
str3=str1+str2
```

2.replication(repeatation)

>creating a new str by combining a str with itself multiple times

```
new_str=str*repeat_times
```

3.f-string

>It is a technique to format a string in multiple ways

using var & expr:

```
f".....{var}.....{expr}....."
```

using round with float:

```
f".....{var:.2f}...."        #round to 2 decimal places
f".....{var:.5f}...."        #round to 5 decimal places
```

using round with padding spaces

```
f"...{var:3}".....#padding with spaces to make 3 chars wide
```

using round with padding zeros

```
f"...{var:03}".....        #padding with zeros to make 3 chars wide
```

4.Iterable

>We can access elements of string using for loop

```
for var in str:
    print(var)
```

4.Indexing

>It is a technique to identify a char of string

>Python provides 2 types of indexing

A. +ve indexing --> 0 to N-1 in forward direction

B. -ve indexing --> -1 to -N in reverse direction

5.Subscript operator

>It is used to get a char of string using index

syntax

```
char=str[index]
```

Note:it raise index out of range error if index is out of range

6.Slicing

>It is a technique to get substring from a string

syntax

```
substr=str[start_index : stop_index : step]
```

start_index----->inclusive

stop_index----->exclusive

step

+ve--->forward slicing

-ve---->reverse slicing

0------>error

7.len()

>It is function that returns no. of chars present in given string

```
no_chars=len(str)
```

String Methods

8.upper()

>It creates a new string by changing letters to upper case

```
new_str=str.upper()
```

9.lower()

>It creates a new string by changing letters to lower case

```
new_str=str.lower()
```

10.capitalize()

>It creates a new string by changing first letter to upper case & rest in lower case

```
new_str=str.capitalize()
```

11.title()

>It creates a new string by changing first letter of each word to upper case & rest in lower case

new_str=str.title()

12.swapcase()

>It creates a new string by exchanging letters to upper & lower case

new_str=str.swapcase()

13.replace()

>It creates a new string by replacing old substr to new substr

new_str=str.replace(old substr,new substr) #all replcements

new_str=str.replace(old substr,new substr,no_replacements) #only given replcements

14.count()

>It returns occurrences of given substr

occur=str.count(substr)

15.strip()

>It creates a new string by removing leading & trailing whitespaces(space,\t,\n).

new_str=str.strip()

16.lstrip()

>It creates a new string by removing only leading whitespaces(space,\t,\n).

new_str=str.lstrip()

17.rstrip()

>It creates a new string by removing only trailing whitespaces(space,\t,\n).

new_str=str.rstrip()

18.startswith()

>It returns True if string starts with given substr,else False

bool=str.startswith(substr)

19.endswith()

>It returns True if string ends with given substr,else False

bool=str.endswith(substr)

Note:to search substr anywhere ,use membership operator(in keyword)

bool=substr in str

20.isupper()

>It returns True if given str in upper case

```
bool=str.isupper()
```

21.islower()

>It returns True if given str in lower case

```
bool=str.islower()
```

22.index()

>It returns lowest index of given substr,error if not found

```
index=str.index(substr)
```

23.rindex()

>It returns highest index of given substr,error if not found

```
index=str.rindex(substr)
```

datatypes
=========

1. Scalar datatypes
   >store 1 value at a time
   exp:int,float,bool,str

2. Container/Collection datatypes
   >can store multiple values at a time
   exp:list,tuple,set,frozenset,dict,etc.

list datatype

>allows duplicate values
>maintain values in the insertion order
>iterable & supports indexing
>mutable
>allows homogenuous(same type values) or hetrogeneous(different type values) values

syntax

```
list_var=[]
list_var=[value1, value2, value3, .....]
```

operations

1.len()

>it is a fuction that returns no. of items in a list

```
items=len(list)
```

2.sum()

>it is a function that returns arithmetic sum of list items

```
result=sum(list)
```

3.min()

>it is a function that returns minimum value of list

```
min_value=min(list)
```

4.max()

>it is a function that returns maximum value of list

```
min_value=max(list)
```

5.sorted()

>it is a function that returns a new list in sorted order

```
asc_list=sorted(list)
desc_list=sorted(list,reverse=True)
```

Note:sum,min,max,sorted operations may fail if values are hetrogenuous & not comparable.

6.concatenation

>it creates a new list by combining lists

```
new_list = list1 + list2
```

7.repititions

>it creates a new list by combining a list with itself multiple times

```
new_list=list*repeat
```

8.append()

>it is a method used to add a value to the last of list

```
list.append(value)
```

9.extend()

>it is a method used to add multiple values in same order to the last of list

```
list.extend([value1,value2,....])
```

10.insert()

>it is a method used to insert a value at given index

```
list.insert(index,value)
```

Note:

if we need to update a value,use assignment operator

```
list[index]=value
```

if we need to update multiple values,use slicing with assignment operator

```
list[start:stop:step]=[value1,value2,....]
```

11 deleting value

A. `value=list.pop(index)  #delete and return value from given index,default index=-1`
B. `del list[index]  #delete value from given index,no default index`
C. `list.remove(value)  #delete value only from lowest index`
D. `while value in list:  #delete all occurrences of value`
   `li.remove(value)`
E. `list.clear()  #delete all values from list and make the list empty`

12.list()

>it is a function used to create empty list or from iterable

```
list_var=list()            #empty list
list_var=list(iterable)    #from iterable
```

13.comprehension

>it is a technique to create a list using expression & for loop in a single statement

```
list_var=[expression for]
```

14.2d list

>it is a list that contains nested lists as values

>it is generally used to represent structured(row,col) data,like student info,emp info etc.

```
list_var=[[v1,v2,...] , [v1,v2,...] , .....]
```

syntax to access value of nested list

```
list_var[2d_index][nested_index]
```

tuple datatype

>It is immutable(unmodifiable) version of list.

syntax

```
tup_var=()
tup_var=(value1,)
tup_var=(value1,value2,.....)
```

operations

1.len()
>it is a fuction that returns no. of items in a tuple
```
items=len(tuple)
```

2.sum()
>it is a function that returns arithmetic sum of tuple items
```
result=sum(tuple)
```

3.min()
>it is a function that returns minimum value of tuple
```
min_value=min(tuple)
```

4.max()
>it is a function that returns maximum value of tuple
```
min_value=max(tuple)
```
5.sorted()
>it is a function that returns a new tuple in sorted order
```
asc_list=sorted(tuple)
desc_list=sorted(tuple,reverse=True)
```

Note:sum,min,max,sorted operations may fail if values are hetrogenuous & not comparable.

6.concatenation
>it creates a new tuple by combining tuples
```
new_tup = tup1 + tup2
```

7.repeatations
>it creates a new tuple by combining a tuple with itself multiple times
```
new_tuple=tuple*repeat
```

8.searching
>membership operator is used to search a value in tuple
```
bool=value in tuple
```

9.tuple()

>it is a function that creates a new empty tuple or from iterable

```
tup_var=tuple()              #empty tuple
tup_var=tuple(iterable)      #from iterable
```

diff b/w list and tuple
---

list	tuple
1.mutable	1.immutable
2.use more memory than tuple	2.use less memory than list
3.slower iteration than tuple	3.faster iteration than list

set datatype
==========

>It is unordered mutable collection of unique values.
>It is not subsctiptable(does not support indexing & slicing) but iterable.

syntax

```
set_var={v1,v2,....}
```

operations

#len,min,max,sum,sorted,sum,searching work same as list

add()

>It is a method used to add a value at arbitrary position

```
set.add(value)
```

remove()

>It is a method used to remove given value,error if not found

```
set.remove(value)
```

pop()

>It is a method used to remove & return arbitrary value of set

```
value=set.pop()
```

union()

>It is a method used to create a new set by combining elements of 2 sets

```
set3=set1.union(set2)
or
set3=set1 | set2  (bitwise or)
```

intersection()

>It is a method used to create a new set by taking common elements of 2 sets

```
set3=set1.intersection(set2)
or
set3=set1 & set2  (bitwise and)
```

difference()

>It is a method used to create a new set by removing elements of set2 from set1

```
set3=set1.difference(set2)
or
set3=set1-set2    (arithmetic subtraction)
```

symmetric_difference()

>It is a method used to create a new set by combining unique elements of 2 sets

```
set3=set1.symmetric_difference(set2)
or
set3-set1 ^ set2  (bitwise xor)
```

set()

>It is a function used to create empty set or from iterable

```
set_var=set()            #empty set
set_var=set(iterable)  #from iterable
```

comprehension

>It is a concise(short & clear) way to create a set using expression & for loop in a single line statement

```
set_var={expression for}
```

frozenset datatype

==========

>It is immutable version of set

syntax

```
fs_var=frozenset()              #empty
fs_var=frozenset(iterable)      #from iterable
```

Unpacking of containers:
==========

>It is a technique to store individual elements of a container to multiple variables
>It is commonly used with parallel iterations and dict .

syntax-1

```
var1,var2,var3,.....var-n=container of n elements
```

syntax-2

```
var1,var2,*var3=container of n elements
```

Note:
>starred expression is unpacked in the last as list
>multiple starred expression are not allowed

Parallel Iterations:
==========

>It is a technique to iterate multiple iterables parallely
>zip() function takes multiple iterables as arg and returns a tuple on each iteration

zip() with for:

```
for tup in zip(iterable1,iterable2,.......):
    code
```

>we may also unpack this tuple

```
for var1,var2,...in zip(iterable1,iterable2,....):
    code
```

Note:
>we may also use zip() without for loop
>zip() works according to shortest length iterable

dict datatype
==========

>A dict is a collection of key value pairs where each key value pair is known as item
>It is mutable & maintains order of items in the insertion order(officially from 3.7 version)
>Keys are unique & duplicate values are allowed
>It is iterable(default keys)

syntax:

```
dict_var={}
dict_var={key : value, key : value, .....}
```

operations

1.Iteration

A. keys

```
for k in dict_var:
    print(k)

or

for k in dict_var.keys():
    print(k)
```

B. values

```
for v in dict_var.values():
    print(v)
```

C. items

```
for tup in dict_var.items():
    print(tup)

or

for k,v in dict_var.items():
    print(k,v)
```

2.Searching

```
bool=key in dict_var                    #search key
bool=value in dict_var.values()         #search value
```

similarly we may implement len,min,max,sorted,sum (sum fails with str)

3.Get a value

A. value=dict_var[key] #error if key not found

B. value=dict_var.get(key,'alternate value') #alternate value if key not found

Note:

keys in dict must be immutable and values may be of any datatype.

4.Nested dict:

>It is a dict that is present as value of an existing dict

exp:

dict_var={key: {key:value,key:value...} }

4.Add/Update Item

dict_var[key] = value

if key already exists,update it's value

if key does not exist,add new item to the last of dict

5.Delete Item

A. del dict_var[key] #remove item of given key,error if key not found

B. value=dict_var.pop(key) #remove item of given key and also return it's value,error if ...

C. key,value=dict_var.popitem() #remove last item and return it's key & value,error if empty dict

D. dict_var.clear() #remove all items & make the dict empty

6.dict()

>this function is used to create empty dict or from iterable of iterables with zip

A. dict_var=dict() #empty dict

B. dict_var=dict(zip(iterable-key,iterable-value))

7.dict comprehension

>it is a concise way to create dict using expression and loop

dict_var={key_expr:value_exp for}

Note:

keys in dict must be immutable and values may be of any datatype.

Function

========

>A function is a named block that provides reusability of logic anywhere in same or other program.

>There are 2 parts of a function

A. function definition --->we define logic

B. function calling --->we run the function

syntax to define fun

```
def funname():
    logic
```

syntax to call a fun

```
funname()
```

Key points

>funname must follow same rules as variable

>we must define function first then call

>if we define same fun multiple times,only latest definition is executed

>functions are executed in the order they are called not they are defined

>built-in functions are thise function defined in library by PSF

>functions that we define in our program,called user defined or custom functions

>there is no concept of entry point fun main() like in C/C++/java.

Local & Global Variables

Local:

>defined inside a function

>can not be used outside function block

>for each call,memory is allocated and destroyed when definition is completed

>2 functions may define local variables of same name

Global:

>defined at top level

>can be used anywhere in program

>memory is allocted only once and destroyed when program is completed

>function may also define global variable using global keyword

exp:

```
a=1             #global
```

```
def show():
    b=2     #local
    print(a)
    print(b)
show()
print(a)
print(b)#error
```

Types of Args

1.Positional Args
2.Keyword Args
3.Default Args
4.Variable length positional
5.Variable length keyword

Positional Args

>These args are passed according to parameter position,it means 1st arg is passed to 1st parm 2nd arg is passed to 2nd param and so on.

>It is also default approach in python to pass args

exp:

```
def show(a,b):
    code

show(10,20)
```

Keyword Args

>These args are passed according by using parameter name
>Improve readability of fun call
exp:

```
def show(a,b):
    code

show(a=10,b=20)
```

Rules:

>keyword arg can not be repeated

>keyword arg must be passed using param name only
>we may mix both positional & keyword and first we need to pass positional then keyword

Default Args

>These args are defined at the time of fun definition
exp:

```
def show(a,b=0):
    code
```

Rules:

>if we pass arg during call to such param,default arg is overriden
>in definition,first we need to define params without default then params with default

Variable length positional

>if a function define parameter with prefix *,such param may accept 0 to N no. of positional args
>internally,it is implemented as tuple
exp:

```
def show(*args):
    code
show()
show(10)
show(10,20,30,.....)
```

Rules:

>only 1 var-len arg is allowed in a fun
>all non default args are defined before it and all default args are defined after it

exp:

```
def print(*args,sep=" ",end="\n"):
    code
```

Variable length keyword

>if a function define parameter with prefix **,such param may accept 0 to N no. of keyword args

>internally,it is implemented as dict
>keyword args can be pass using any name

exp:

```
def show(**args):
    code
show()
show(a=10)
show(a=10,b=20,c=30,.....)
```

Rules:

>only 1 var-len arg is allowed in a fun
>if you mix both var len positional & var len keyword,first define var len positional then keyword

exp:

```
def show(*arg,**kwargs):
    code
```

return keyword

>it is used to return local data of a fun to caller
>it is the last stmt of function block
>if we do not define return stmt,default return will be None

exp:

```
def fun():
    ..
    ..
    return value
```

>we may also return multiple values

```
def fun():
    ..
    ..
    return v1,v2,... #default tuple ,you may also use list,set,dict
```

>we may also use annotation of return type

exp:

```
def fun()->type:
    ..
    ..
```

return value

Module
=======

>A module is a python file that contains functions,variables & classes.
>We use module by importing it into another python file

Note:Script is a also a python file that we run directly & Module is a python file that we run using import

syntax to import module

```
import module
module.fun()
```

syntax to import module with alias

```
import module as alias
alias.fun()
```

syntax to import a fun from module

```
from module import fun
fun()
```

__name__

>It is predefined variable available to each python file with following 2 possible values

__main__	if python file is executed as script
filename	if python file is imported

>we may use this __name__ in module to not to import some stmts
exp:

```
code that you want to import

if __name__=="__main__":
    code that you do not want to import
```

Package
=======

>A package is a collection of similar type modules
>A package organize modules in better way than otherwise
>A package also provides identity to a module that helps to resolve module name ambiguity

>Physically,A package is just a folder with name of package and contains modules

syntax to import module from package

```
import package.module
package.module.fun()

or

from package import module
module.fun()
```

syntax to import module from package with alias

```
import package.module as alias
alias.fun()

or

from package import module as alias
alias.fun()
```

syntax to import a fun from package

```
from package.module import fun
fun()
```

math module

>It provides functions realted to mathematical operations

1.sqrt()

>it returns square root of given num

```
res=math.sqrt(num)
```

2.cbrt()

>it returns cube root of given num

```
res=math.cbrt(num)
```

3.pow()

>it represents functionality of exponent operator

```
res=math.pow(a,b)
```

4.log()

>It returns log value of a number on a base

```
res=log(number,base)
```

5.dist()

>It returns Euclidean distance between 2 points

```
res=math.dist(p1,p2)
p1,p2 can be iterable like tuple/list
```

6.exp()

>It returns exponential value of a num

```
res=math.exp(num)
```

etc.

Note:math module also provides some variables(attributes/properties) like

math.pi---->float--->3.141592653589793
math.inf--->float--->infinity
math.e----->float--->2.71...
math.nan-->float-->not a number
etc.

random module

>It provies functions related to random number generation

1.randint()

>It returns a random int value in given range

```
res=random.randint(a,b)
```

2.uniform()

>It returns a random float value in given range

```
res=random.uniform(a,b)
```

3.shuffle()

>It makes the list unordered

```
random.shuffle(list)
```

4.choice()

>It returns a value from the given sequence

```
res=random.choice(seq)
```

5.choices()

>It returns k values from the given sequence with replacement(may pick same value morethan once)

```
res=random.choices(seq,k)
```

6.sample()

>It returns k values from the given sequence without replacement(a value is picked only once)

etc.

```
res=random.sample(seq,k)
```

time module

>It provides functions related to date & time activities

1.sleep()

>this function used to pause execution of program till given secs

```
time.sleep(secs)
```

2.ctime()

>this function returns system date & time in a pre-defined format

```
default_format=time.ctime()
```

3.strftime()

>this function returns system date & time in given format

```
user_format=time.strftime("format")
```

>to define format,we use following directives

%a day name in short
%A day name in full
%b month name in short
%B month name in full
%d date
%m month in digit
%y year without century
%Y year with century
%r time in 12 hrs format with AM PM

etc.

4.perf_counter()

>this function is used to measure how long something takes to run.

>You note the time before your code runs and after, then subtract.

>Very high precision (more accurate than normal time)

exp:

```
import time
start = time.perf_counter()

# code to measure
for i in range(1000000):
    pass

end = time.perf_counter()

print(end - start)
```

etc.

sys module

>It provides functions related to interpreter activities performed at runtime

1.version

>this attribute provides version information about executing interpreter

```
version_info=sys.version
```

2.maxsize

>this attribute provides information about max possible index in python

```
max_index=sys.maxsize
```

Note:It is theoritically possible but may not be practically,also depends on platform

3.path

>this attribute provides information about directories(folders) where interpreter search a module

```
list=sys.path
```

4.prefix

>this attribute provides information about Python Installation directory

5.platform

>this attribute provides information about os

6.implementation

>this attribute provides information about interpreter(cpython,jython,...)

7.getsizeof()

>this function returns size of python object in bytes

```
size_bytes=sys.getsizeof(object)
```

8.exit()

>this function terminates python interpreter

```
sys.exit()
```

9.stdout.write()

>this function actually writes str data to standard output

```
sys.stdout.write(str_data)
```

Note:print() internally uses this function

etc.

os module
---

this module provides functions related to file & folder(directory) activities

1.getcwd()

>it return path of current working directory(cwd)

```
path=os.getcwd()
```

2.listdir()

>it returns items list present at given path

```
list_items=os.listdir("path")
```

Note:if path is not given,default to cwd

how to write path
---

3 ways:

A. "drive:/folder/folder/....."
B. "drive:\\folder\\folder\\...."
C. r"drive:\folder\folder\....."

3.rename()

>it renames a file or directory

```
os.rename(src_path,dest_path)
```

4.remove()

>it removes a file permanently

```
os.remove('file_path')
```

5.rmdir()

>it removes a folder permanently

```
os.rmdir('folder_path')
```

>to remove directory tree,use shutil module

```
import shutil
shutil.rmtree("folder_path")
```

6.mkdir()

>it creates a directory

```
os.mkdir("dir_name")
```

7.makedirs()

>it creates a directory tree(nested folders)

```
os.makedirs("dir_tree")
```